# Google Workspace Migrate — Deployment & Large-Scale Migration Execution

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

# Contents


Executive Summary 4

Solution Overview 4

What this is . . . 4

How it works (reference architecture) . . . 5

Deployment model . . . 6

Benefits . . . 6

Limitations / scope boundary . . . 7

Requirements 7

R1. Node platform infrastructure (Windows Server hosts + database) . . . 7

R2. Microsoft 365 source app registration + scopes (`Connection – Microsoft 365 – Source`) . . . 8

R3. Google target Cloud project + delegated service account (`Connection – Google Workspace – Target`) . . . 8

R4. Network path, bandwidth, and firewall egress . . . 8

R5. Source inventory / discovery baseline . . . 9

R6. Mapping and identity-bridge source data (`Mapping – ... `, `Bridge – Identity – Corp`) . . . 9

R7. Migration service accounts and least-privilege operations . . . 9

Global Requirements — Google Workspace & Gemini Solutions 10

1. Google Workspace / Cloud Identity tenant . . . 10

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 10

3. DNS control for the routed domains . . . 11

4. Identity & provisioning source of truth . . . 11

5. A pilot scope (OU / group) and change control . . . 11

6. Security, compliance & governance stakeholders . . . 11

Implementation 12

Phase 1 — Assess . . . 12

Phase 2 — Design . . . 13

Phase 3 — Configure . . . 13

Phase 4 — Pilot (pilot-first, on `Migration Wave – Finance – Pilot`) . . . 14

Phase 5 — Validate . . . 15

Phase 6 — Expand . . . 15

Testing & Validation 15

T1. Node platform health (Gate C) . . . 16

T2. Connection authentication (Gate C) . . . 16

T3. Mapping & bridge integrity (Gate C) . . . 16

T4. Pilot scan review (Gate C) . . . 16

T5. Pre-stage throughput & throttling (Gate D) . . . 16
T6. Delta correctness (Gate D) . . . 17
T7. Item-count parity (Gate E) . . . 17
T8. Permission & ownership fidelity (Gate E) . . . 17
T9. Mail count & flow parity (Gate E, mail waves) . . . 17
T10. End-to-end pilot sign-off (Gate D→E) . . . 18
Troubleshooting 18
Throttling / API rate limits . . . 18
Connection authentication / permission errors . . . 18
Mapping / identity-bridge errors . . . 19
Delta pass re-copies everything, or misses changes . . . 19
Incomplete / skipped / unsupported items . . . 19
Node platform / database issues . . . 19
Bandwidth / network egress bottleneck . . . 20
Mail routing / cutover problems . . . 20
Permissions/sharing dropped or over-broadened on target . . . 20
Best Practices 20
Platform & sizing . . . 20
Connections & identity . . . 21
Waves, throttling & sequencing . . . 21
Validation & change control . . . 21
Safety & hygiene . . . 22
Deliverables Summary 22
Documentation . . . 22
Python tools (5) — Google Workspace Admin SDK / Microsoft Graph, safe-by-default . . . 22
Packaged outputs . . . 23
Appendix A — PowerShell Scripts 24

# Executive Summary

**Outcome.** The organization can move enterprise-scale **files and mail** from **Microsoft 365** (SharePoint Online, OneDrive, Exchange Online) into **Google Workspace** (Drive, Shared Drives, Gmail) using **Google Workspace Migrate**, Google's own large-scale migration platform — deployed, mapped, throttled, waved, and validated so large volumes move **reliably** and with **preserved ownership and sharing**.

**Approach.** This is a **tool deployment-and-execution playbook**. It stands up the self-hosted node platform (management/database node plus scalable worker nodes), wires the **Microsoft 365 and Google connections**, builds **mappings and the identity bridge**, and runs data in **throttled waves** using the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. A small named **pilot wave** is migrated end to end — pre-stage bulk pass, delta pass, then gated cutover — and validated for item-count, permission, and mail parity **before** any production wave runs. Five **Python tools** (Google Workspace Admin SDK and Microsoft Graph) handle the scriptable edges: node sizing from a real source baseline, mapping/connection artifact generation, wave batching with a throttling plan, live migration health/throttling monitoring, and post-migration validation reporting. Every tool is **safe-by-default** (read-only or dry-run until explicitly applied) with **no credentials in code**; the tool's deep console internals are **deferred to Google's own documentation** and version-specific behavior is flagged **"verify for your version."**

**Why it is low-risk.** Nothing scales until a pilot passes explicit **pass/fail gates**. The bulk of the data moves while users keep working on the source; only **cutover** is high-impact, and it runs inside a change window with a **tested, reversible rollback** (routing reverts, source data is preserved). Throttling is planned conservatively and calibrated from measured pilot throughput, so migrations run steadily under source/target API limits rather than tripping them. The result is a predictable, auditable, enterprise-scale migration on Google's native tool — for a fraction of the cost of an open-ended consulting engagement.

# Solution Overview

## What this is

This solution is the **deployment and large-scale execution playbook** for **Google Workspace Migrate (GWSMigrate)** — Google's own on-premises platform for moving enterprise volumes of files and mail from **Microsoft 365** (SharePoint Online, OneDrive for Business, Exchange Online) into **Google Workspace** (Drive, Shared Drives, Gmail). It covers everything an enterprise migration team must own to run the tool at scale: designing and sizing the **node/database platform**, wiring the **Microsoft 365 and Google connections**, building **mappings and bridges**, sequencing **throttled migration waves**, and **validating** the results.

It is a **tool playbook**, not a strategy paper. It gives you the architecture, prerequisites, setup order, mapping and wave design, throttling levers, and validation gates you need to run Google Workspace Migrate to implement. Because the tool's console changes across releases, the **deep console internals, exact screen labels, licensing, and support boundaries are deferred to Google's own Google Workspace Migrate documentation** — where a step depends on version-specific behavior it is flagged **"verify for your version."** The *procedure, sequencing, and design* are given here; the *button-by-button console detail* you confirm against Google's current docs.

### *Where it sits among the migration solutions*

• **The tool-agnostic program/strategy playbook** owns the *decision* of how to migrate (waves, coexistence, cutover strategy, org change) independent of tool choice. This solution assumes that decision is made and executes it on Google's native tool.

• **The third-party tool playbooks** (BitTitan MigrationWiz, CloudM Migrate, and the Google Workspace Migration for Microsoft Exchange / GWMME desktop tool) cover *those* tools. This solution is specifically for **Google Workspace Migrate**, the server-based platform Google provides for **large-scale file and mail** moves.

• Use Google Workspace Migrate when the dataset is large enough (typically SharePoint/OneDrive at scale plus mail) to justify standing up node infrastructure and running parallelized, throttled waves under central control.

## How it works (reference architecture)

Google Workspace Migrate is a **self-hosted platform** the organization runs on Windows Server hosts inside its own network. Its logical parts:

• **Platform node (management node)** — runs the management service, the web console the migration team drives, and the encrypted **platform database** that holds project state, mappings, scan results, and run history. Named here `GWSM Platform Node – Migration – Prod`.

• **Worker (data) nodes** — the horsepower. Each worker node runs the scan and transfer work in parallel; you add nodes to increase throughput. Named `GWSM Worker Node – Migration – NN` (e.g. `01`, `02`).

• **Connections** — credentialed links to each side: a **Microsoft 365 connection** (app registration in Microsoft Entra ID with Graph/SharePoint scopes) named `Connection – Microsoft 365 – Source`, and a **Google connection** (a Google Cloud project + a service account with domain-wide delegation) named `Connection – Google Workspace – Target`.

• **Mappings** — how a **source object maps to a target object**: user → user, OneDrive → My Drive, SharePoint site/library → Shared Drive, mailbox → Gmail. Built from a **mapping CSV** the team authors and imports. Named per scope, e.g. `Mapping – Finance OneDrive – Pilot`.

• **Bridges** — the connective identity mapping that resolves **source identities to target identities** (and permissions) so ownership and sharing survive the move. Named `Bridge – Identity – Corp`.

• **Scans then migrations** — you **scan** first (read-only discovery of what exists and how big), review the scan, then run a **migration** for a defined scope. Every run is a **wave**.

The team drives all of this from the platform node's console; the **Python tools in this solution** work *alongside* it against the **Google Workspace Admin SDK and Microsoft Graph** to size nodes, generate the mapping/connection artifacts the tool consumes, plan throttled waves, and validate the target after each wave — the parts that are scriptable outside the tool's own console.

### *The wave model*

Data moves in **waves** — a scoped batch (a set of users/sites) migrated together:

1. **Pre-stage (bulk) pass** — the large, slow copy of the bulk of the data while users still work on the source. Run with conservative throttling over days/nights.
1. **Delta pass(es)** — incremental re-runs that carry only what changed since the last pass.
2. **Cutover** — the final delta plus the switch of the user to the target (mail routing / user communication). High-impact; gated and reversible.

Waves are named `Migration Wave – <Scope> – <Stage>` (e.g. `Migration Wave – Finance – Pilot`, `Migration Wave – Sales – Delta`).

## Deployment model

This solution follows the house model end to end, with explicit pass/fail gates and a documented rollback for every production change:

**Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.**

• **Assess** — inventory the source (item counts, data volume, largest mailboxes/sites, sharing complexity); size the node platform; confirm prerequisites.

• **Design** — node count/sizing, connection scopes, mapping strategy (OneDrive→My Drive, SharePoint→Shared Drive), identity bridge, wave plan and throttling budget.

• **Configure** — stand up nodes + database, create both connections, import mappings and the identity bridge, run a **scan**.

• **Pilot** — migrate a small named wave (`Migration Wave – Finance – Pilot`) pre-stage → delta →
cutover, measuring throughput, error rate, and throttling behavior.

• **Validate** — item-count parity, permission/ownership fidelity, and (for mail) message-count and mail-flow checks against pass/fail gates.

• **Expand** — schedule production waves only after the pilot passes, tuning throttling and node count from the pilot's measured throughput.

## Benefits

• **Google's native large-scale tool**, run correctly: node platform sized to the dataset, waves parallelized and throttled so you don't trip source or target API limits.

• **Predictable, reversible rollout** — pilot-first with measured throughput and defined gates before any production wave; cutover is the only high-impact step and it is gated and reversible.

• **Fidelity preserved** — identity bridge and mapping design keep ownership and sharing intact; post-wave validation proves item-count and permission parity rather than assuming it.

• **Scriptable edges** — the five Python tools remove the manual, error-prone parts (sizing,

artifact generation, throttling math, validation) while leaving the tool's console authoritative.

### Limitations / scope boundary

- **Tool-console internals are deferred to Google's documentation.** Exact console labels, wizard

screens, node installer prompts, supported source/target scopes, and licensing/support terms change across releases — this playbook gives procedure and sequencing and flags version-specific behavior **"verify for your version."** Always confirm the current supported-scope matrix (what content types Migrate does and does not carry) in Google's docs before committing a wave plan.

- **Not a Teams/Chat data-mover.** This solution scopes **files (SharePoint/OneDrive) and mail**,

the workloads Google Workspace Migrate is built for. Teams/Chat history and other workloads are out of scope here and follow their own approach.

- **Gemini or any preview target-side feature** referenced is flagged **"verify in your tenant."**
- The Python tools are **safe-by-default** (read-only or `--dry-run` until `--apply`) and never

drive an invented tool API — they generate the CSV/JSON artifacts Migrate consumes or read the Admin SDK / Graph to validate; they do not fabricate a private Migrate endpoint.

## Requirements

These are the **solution-specific** prerequisites for deploying and running **Google Workspace Migrate** at scale. They build on the shared **Global Requirements — Google Workspace & Gemini Solutions** (`global-requirements-google.md`): a provisioned **Google Workspace / Cloud Identity tenant with super-admin** (GR-1), administrative access to the source **Microsoft 365 / Entra** environment (GR-2), **DNS control** for routed domains (GR-3), an **identity source of truth** and provisioning model (GR-4), a **pilot OU/group and change control** (GR-5), and engagement with **security/compliance stakeholders** (GR-6). Do not re-verify those here; confirm them, then verify the items below.

> *Where a requirement depends on Google Workspace Migrate's current release (supported node OS, supported source/target scope, sizing guidance),* ***verify against Google's Google Workspace Migrate documentation for your version*** *— those specifics change and are not restated here.*

### R1. Node platform infrastructure (Windows Server hosts + database)

- **Description:** Windows Server hosts to run the **platform node** (management service + web

console + platform database) and one or more **worker nodes**, each meeting Google Workspace Migrate's current CPU/RAM/disk and supported-OS requirements, inside the organization's network.

- **Why required:** Google Workspace Migrate is self-hosted; scan/transfer throughput is a direct

function of node count and node sizing. Under-provisioning is the most common cause of missed migration timelines.

- **How to verify:** Confirm the provisioned hosts meet Google's **current** minimum spec (verify

for your version) and that the platform node's database volume has headroom for project state and scan results across all planned waves. Run `node-sizing-assessment.py` (this solution) against the source

inventory to get a recommended node count for the target throughput.

- **How to implement if missing:** Provision the hosts (physical or IaaS), install the supported OS, and reserve a dedicated encrypted volume for the platform database. Size to the recommendation from `node-sizing-assessment.py`, rounding up for delta passes and headroom.

## R2. Microsoft 365 source app registration + scopes (`Connection – Microsoft 365 – Source`)

- **Description:** An **Entra ID app registration** with the Microsoft Graph / SharePoint application permissions Google Workspace Migrate requires to read Exchange Online, OneDrive, and SharePoint at scale, plus admin consent granted tenant-wide.
- **Why required:** The Microsoft 365 connection authenticates as this app to scan and read source content; missing or unconsented scopes are the top cause of scan/transfer permission failures.
- **How to verify:** In the Entra admin center, confirm the app registration exists, the required application permissions are present, and **admin consent** is granted (green status). Confirm the client secret/certificate is current and stored in a secret manager, not in code.
- **How to implement if missing:** Register the app, add the permissions Google Workspace Migrate's documentation specifies **for your version**, grant admin consent, and record the client ID + secret reference for the connection. Use least privilege the tool supports.

## R3. Google target Cloud project + delegated service account (`Connection – Google Workspace – Target`)

- **Description:** A **Google Cloud project** with the required Workspace APIs enabled (Admin SDK Directory, Drive, Gmail, Groups) and a **service account with domain-wide delegation** authorized for the scopes Migrate uses to write into Drive/Shared Drives and Gmail.
- **Why required:** The Google connection impersonates target users via the delegated service account to create content and set ownership/permissions on the target side.
- **How to verify:** In the Google Admin console under **Security → Access and data control → API controls → Domain-wide delegation**, confirm the service account's client ID is present with the exact OAuth scopes required. Confirm the APIs are enabled in the Cloud project.
- **How to implement if missing:** Create the Cloud project, enable the APIs, create the service account, and authorize its client ID for the required scopes in Domain-wide delegation. Store the key in a secret manager; **never commit the key**.

## R4. Network path, bandwidth, and firewall egress

- **Description:** Reliable, sized network egress from the node platform to **both** Microsoft 365 and Google endpoints, with any required firewall/proxy allow-listing, and enough sustained bandwidth to move the assessed data volume within the migration window.

• **Why required:** Every byte transits the nodes; bandwidth and firewall egress cap real throughput regardless of node count.

• **How to verify:** Confirm the platform/worker nodes can reach the Microsoft and Google service endpoints (verify the current endpoint/URL list for your version) through the corporate proxy/firewall. Compare available sustained bandwidth against the assessed volume ÷ window.

• **How to implement if missing:** Allow-list the required egress, provision bandwidth (or place nodes where bandwidth is best), and schedule bulk pre-stage passes for off-peak windows.

## R5. Source inventory / discovery baseline

• **Description:** A **quantified inventory** of the source: user/mailbox counts and sizes, OneDrive/SharePoint site and library counts, item counts, largest objects, and sharing/permission complexity — the baseline every wave is validated against.

• **Why required:** Node sizing, wave scoping, throttling budgets, and validation gates all key off real numbers. Without a baseline you cannot prove parity after a wave.

• **How to verify:** Run `node-sizing-assessment.py` (reads Microsoft Graph, read-only) to produce the inventory and volume/throughput estimate; sanity-check against the Microsoft admin center reports.

• **How to implement if missing:** Grant the assessment app read scopes and run the discovery before designing waves; store the baseline as the validation reference.

## R6. Mapping and identity-bridge source data (`Mapping – … ,Bridge – Identity – Corp`)

• **Description:** Authoritative **source→target mapping data** (which source user/site maps to which target user/Shared Drive) and the **identity bridge** data that resolves source identities to target identities so ownership/sharing survive.

• **Why required:** Mappings and the bridge are what Google Workspace Migrate consumes to route content and preserve permissions; wrong or incomplete mappings misplace data or drop sharing.

• **How to verify:** Run `connection-mapping-generator.py --dry-run` (this solution) to produce and validate the mapping CSV + bridge from the directory data; confirm every in-scope source object resolves to exactly one target.

• **How to implement if missing:** Reconcile the directories (see GR-4 source of truth), then generate the mapping/bridge artifacts and review before import.

## R7. Migration service accounts and least-privilege operations

• **Description:** Dedicated, named service identities for the migration platform (the M365 app, the

Google delegated service account) with least privilege, plus a defined **change window** for cutover waves (mail routing changes are high-impact — GR-3/GR-5).

• **Why required:** Migration runs with powerful cross-tenant access; scoping it to dedicated, auditable identities and gating cutover under change control limits blast radius.

• **How to verify:** Confirm the identities are dedicated (not a person's admin account), documented, and revocable; confirm a change/approval path exists for cutover.

• **How to implement if missing:** Create the dedicated identities, document them, and agree the cutover change window and rollback with the owners before scheduling production waves.

# Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

## 1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

## 2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change

the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

## 3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

## 4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

## 5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model;

the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the

owners before making production changes.

## 6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

# Implementation

Run these steps in order. They follow the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback for every production change. Every artifact is named with the convention `<Purpose> – <Scope> – <Stage>`; reuse the exact names in testing, troubleshooting, and the scripts.

> ***Tool-console boundary.*** *Where a step drives Google Workspace Migrate's own console (node installer prompts, connection wizard, scan/migration screens), the **procedure and sequence** are given here but the **exact screen labels and version-specific options are deferred to Google's Google Workspace Migrate documentation** — confirm them **for your version** as you go. The Python tools operate outside the console against the Admin SDK / Graph and generate the artifacts the console imports.*

## Phase 1 — Assess

### *1.1 Baseline the source (read-only)*

Run the discovery to quantify the migration and feed sizing:

```
python node-sizing-assessment.py --graph --out baseline-source.json
```

This reads Microsoft Graph (read-only) for mailbox/OneDrive/SharePoint counts, sizes, largest objects, and sharing complexity, and prints a recommended **worker-node count** for a target throughput and window. Store the output as `baseline-source.json` — it is the reference every validation gate compares against.

### *1.2 Confirm prerequisites*

Verify R1–R7 (Requirements) and the shared GR-1…GR-6. Do not proceed until the node hosts (R1), both connection identities (R2/R3), network egress (R4), and the pilot scope (GR-5) are confirmed.

**Gate A (enter Design):** baseline captured; node hosts provisioned; both connection identities created and consented; pilot OU/group named.

## Phase 2 — Design

### *2.1 Size the platform*

Using `baseline-source.json`, fix the node platform:

- `GWSM Platform Node – Migration – Prod` — one management/database node (verify current spec).
- `GWSM Worker Node – Migration – NN` — worker nodes at the count `node-sizing-assessment.py` recommends for the throughput needed to finish within the window (round up for delta passes).

### *2.2 Design mappings and the identity bridge*

Decide the target shape and generate the artifacts (dry-run):

```
python connection-mapping-generator.py --dry-run \
    --graph --directory --out mapping-plan/
```

Standard mapping design:

- **OneDrive** → **My Drive** per user (`Mapping – <Dept> OneDrive – <Stage>`).
- **SharePoint site/library** → **Shared Drive** (`Mapping – <Site> SharePoint – <Stage>`).
- **Mailbox** → **Gmail** per user (`Mapping – <Dept> Mail – <Stage>`).
- `Bridge – Identity – Corp` — resolves every source identity to a target identity so ownership and sharing survive. Every in-scope source object must resolve to exactly one target.

### *2.3 Plan waves and the throttling budget*

Group in-scope users/sites into waves and compute a safe throttling budget:

```
python wave-batch-builder.py --dry-run \
    --baseline baseline-source.json --wave-size 250 --out waves/
```

This emits per-wave batch files (`Migration Wave – <Scope> – <Stage>`) and a **throttling plan** (concurrency per node, requests/sec ceilings) sized to stay under source/target API limits. Start conservative; the pilot will calibrate the real ceiling.

**Gate B (enter Configure):** node count fixed; mappings + `Bridge – Identity – Corp` generated and reviewed (every object resolves once); wave plan + throttling budget drafted.

## Phase 3 — Configure

### *3.1 Stand up the node platform*

1. Install Google Workspace Migrate on `GWSM Platform Node – Migration – Prod` (follow Google's node installer for your version — screen labels deferred to Google's docs). Point it at the dedicated encrypted database volume (R1).
1. Install and register each `GWSM Worker Node – Migration – NN` to the platform node.

2. Confirm all nodes report healthy in the console before creating connections.

### *3.2 Create the connections*

1. `Connection – Microsoft 365 – Source` — enter the Entra app registration (R2) client ID + secret reference and the scopes; test the connection until it authenticates.
1. `Connection – Google Workspace – Target` — enter the Google delegated service account (R3); test until it authenticates and can impersonate a target user.

Validate connection health before importing anything:

```
python migration-health-check.py --connections --dry-run
```

### *3.3 Import mappings and the bridge*

Import the reviewed `mapping-plan/` CSVs and `Bridge – Identity – Corp` into the project (console import — verify the import screen for your version). Re-run the generator with `--validate` to confirm the imported set matches what was generated.

### *3.4 Run a scan (read-only discovery)*

Run a **scan** over the pilot scope only. A scan is read-only: it discovers what exists and how big, with no writes to the target. Review the scan for unsupported items, oversized objects, and permission anomalies before any migration.

**Gate C (enter Pilot):** all nodes healthy; both connections test-pass; mappings + bridge imported and validated; pilot scan complete and reviewed with no blocking anomalies.

## Phase 4 — Pilot (pilot-first, on `Migration Wave – Finance – Pilot`)

Run the pilot wave through the full pre-stage → delta → cutover sequence on the named pilot scope (GR-5). **Rollback** for each step is defined below.

### *4.1 Pre-stage (bulk) pass*

Start the migration for `Migration Wave – Finance – Pilot` with the throttling plan from Phase 2. Users keep working on the source; nothing is cut over. Monitor throughput and error rate:

```
python migration-health-check.py --wave "Migration Wave - Finance - Pilot" --watch
```

**Rollback (pre-stage):** the source is untouched and the target is not yet live to users — pause or stop the run, delete the partially-migrated target content for the wave, correct the mapping or throttling, and re-run. No user impact.

### *4.2 Delta pass*

Re-run the wave in **delta** mode to carry only what changed since the pre-stage pass. Confirm the delta completes cleanly and the incremental item count is small and expected.

### *4.3 Cutover (high-impact — gated)*

Only inside the agreed change window (R7/GR-5):

1. Run the **final delta**.

2. Switch the pilot users to the target per the routing/communication plan (mail routing is the

high-impact change — coordinate DNS/routing under change control, GR-3).

**Rollback (cutover):** if validation (Phase 5) fails, revert mail routing to the source under the same change window, keep users on the source, and treat the migrated target content as staging until re-validated. Because routing is reversible and the source data is preserved through cutover, the wave can be re-run.

**Gate D (Pilot passes → enter Validate/Expand):** pre-stage + delta + cutover completed for the pilot wave; error rate within threshold; throttling stable (no sustained source/target throttling).

## Phase 5 — Validate

Validate the pilot against the baseline before scaling. Run the post-migration validation report:

```
python post-migration-validation.py \
    --wave "Migration Wave - Finance - Pilot" \
    --baseline baseline-source.json --out validate-pilot/
```

It compares **target item counts vs. source baseline**, checks **ownership/permission fidelity** (sharing preserved via `Bridge – Identity – Corp`), and, for mail, checks **message-count parity and basic mail-flow**. See `05-testing.md` for the pass/fail criteria of each gate.

**Gate E (enter Expand):** item-count parity within threshold; permissions/ownership preserved; mail count + flow pass. Any gate failure → do not expand; fix and re-validate.

## Phase 6 — Expand

Only after the pilot passes Gate E:

1. Tune **node count** and the **throttling plan** using the pilot's measured throughput

(`wave-batch-builder.py --calibrate --from validate-pilot/`).

1. Schedule production waves (`Migration Wave – Sales – Prod`, etc.) in dependency order, each with

the same pre-stage → delta → cutover sequence, gates, and rollback.

1. Validate **every** production wave with `post-migration-validation.py` before closing it; keep

`migration-health-check.py --watch` running through each wave to catch throttling early.

1. Decommission node infrastructure and revoke the migration service identities (R7) only after the

final wave is validated and closed.

# Testing & Validation

Each test below has an explicit **pass/fail** result. Run them at the gate indicated in `03-implementation.md`. The named artifacts (`Migration Wave – Finance – Pilot`, `Bridge – Identity – Corp`, the connections) are the same ones created during implementation.

## T1. Node platform health (Gate C)

• **Procedure:** In the Google Workspace Migrate console, confirm `GWSM Platform Node – Migration – Prod and every GWSM Worker Node – Migration – NN` report healthy and connected. Confirm the platform database volume has headroom.

• **Pass:** all nodes healthy/connected; database volume < agreed utilization threshold.

• **Fail:** any node unhealthy/disconnected, or database volume near capacity → resolve before scanning.

## T2. Connection authentication (Gate C)

• **Procedure:** Run `python migration-health-check.py --connections --dry-run`, then use the console's built-in connection test for `Connection – Microsoft 365 – Source` and `Connection – Google Workspace – Target`.

• **Pass:** both connections authenticate; the Google connection can impersonate a target user; no scope/consent errors.

• **Fail:** auth or scope error → fix the app registration (R2) or domain-wide delegation (R3) and retest.

## T3. Mapping & bridge integrity (Gate C)

• **Procedure:** Run `python connection-mapping-generator.py --validate --against mapping-plan/`.

• **Pass:** every in-scope source object resolves to exactly one target; `Bridge – Identity – Corp` resolves all source identities; no duplicates or orphans.

• **Fail:** any unresolved, duplicate, or orphaned mapping → correct the source data and regenerate before importing.

## T4. Pilot scan review (Gate C)

• **Procedure:** Review the pilot scan output for unsupported item types, oversized objects, and permission anomalies.

• **Pass:** no blocking unsupported items; oversized objects identified and dispositioned; anomalies understood.

• **Fail:** blocking unsupported/oversized items unaddressed → disposition them (or confirm expected behavior for your version in Google's docs) before migrating.

## T5. Pre-stage throughput & throttling (Gate D)

• **Procedure:** During the pilot pre-stage pass, run `python migration-health-check.py --wave "Migration Wave - Finance - Pilot" --watch`.

• **Pass:** sustained throughput within the planned range; **no sustained source/target throttling** (429/Graph throttling or Google rate-limit responses stay within the plan's ceiling); error rate below threshold.

• **Fail:** sustained throttling or error rate above threshold → lower concurrency in the throttling plan and re-run (see `06-troubleshooting.md`).

## T6. Delta correctness (Gate D)

• **Procedure:** After the delta pass, confirm the incremental item count is small and equals the changes made on the source since pre-stage.

• **Pass:** delta carries only changed items; no full re-copy; no unexpected deletions on target.

• **Fail:** delta re-copies everything or misses changes → verify delta configuration for your version in Google's docs and re-run.

## T7. Item-count parity (Gate E)

• **Procedure:** Run `python post-migration-validation.py --wave "Migration Wave - Finance - Pilot" --baseline baseline-source.json --out validate-pilot/`.

• **Pass:** target item counts (Drive files, Shared Drive contents) match the source baseline within the agreed threshold (accounting for documented unsupported items).

• **Fail:** counts diverge beyond threshold → investigate the delta of missing items in the report and re-run the affected scope.

## T8. Permission & ownership fidelity (Gate E)

• **Procedure:** From the same validation report, review the ownership/permission comparison (sharing preserved via `Bridge – Identity – Corp`).

• **Pass:** owners and sharing on target match the intended source-to-target mapping; no over-permissioning (no unexpected public/broad sharing introduced).

• **Fail:** ownership wrong or sharing dropped/broadened → fix the bridge/mapping and re-run for the affected objects.

## T9. Mail count & flow parity (Gate E, mail waves)

• **Procedure:** From the validation report, compare per-mailbox message counts source vs. target and send a round-trip test to a cutover pilot user.

• **Pass:** message counts match within threshold; test mail delivers to the target mailbox; no

routing loop.

- **Fail:** count mismatch or delivery failure → verify mail mapping/routing and re-run the mail

scope; for routing issues revert routing (cutover rollback) and diagnose.

### T10. End-to-end pilot sign-off (Gate D→E)

- **Procedure:** Confirm T1–T9 all pass for `Migration Wave – Finance – Pilot`.
- **Pass:** all gates green → proceed to **Expand** and schedule production waves.
- **Fail:** any gate red → do not expand; remediate and re-validate the affected test.

## Troubleshooting

Common failure modes running **Google Workspace Migrate** at scale, and how to resolve them. Named artifacts (`Connection – Microsoft 365 – Source`, `Bridge – Identity – Corp`, `Migration Wave – Finance – Pilot`, the throttling plan) are the ones created in `03-implementation.md`.

> *For anything tied to a specific console screen, node installer prompt, or supported-scope behavior,* ***verify against Google's Google Workspace Migrate documentation for your version*** *— the resolutions below are procedural, not version-locked.*

### Throttling / API rate limits

**Symptom:** throughput drops; logs show Microsoft Graph **429 / "throttled"** or Google **rate-limit / quota** responses; `migration-health-check.py --watch` reports sustained throttle events (≥ threshold).

- **Cause:** concurrency/request rate is above the source or target ceiling for your tenant.
- **Resolve:** lower `concurrency_per_node` and the `graph_rps_ceiling` / `google_rps_ceiling` in the

throttling plan (`wave-batch-builder.py`), let the current run drain, and re-run. Prefer running **steadily under** the ceiling over tripping throttling and backing off. Move heavy pre-stage passes to **off-peak** windows. Re-raise ceilings only via `--calibrate` after a clean pilot.

### Connection authentication / permission errors

**Symptom:** `Connection – Microsoft 365 – Source` or `Connection – Google Workspace – Target` fails its test; scan or transfer aborts with auth/scope/consent errors.

- **Microsoft side:** the Entra app registration (R2) is missing a required Graph/SharePoint

application permission or **admin consent** was not granted, or the client secret expired. Re-check the permissions **for your version**, grant admin consent, rotate the secret in the secret manager.

- **Google side:** the service account's client ID is not authorized in **Domain-wide delegation** for

the exact OAuth scopes (R3), a required API is not enabled in the Cloud project, or the service account cannot impersonate the target user. Add the scopes in Admin console → Security → API controls → Domain-wide delegation; enable the APIs. Confirm with `migration-health-check.py`

`--connections`.

## Mapping / identity-bridge errors

**Symptom:** content lands in the wrong target, ownership is wrong, or scan reports unresolved objects.

- **Cause:** a source object doesn't resolve to exactly one target, or `Bridge – Identity – Corp` is missing/duplicated identities.
- **Resolve:** run `connection-mapping-generator.py --validate` — it flags unresolved, duplicate, or orphaned identities. Fix the directory source of truth (GR-4), regenerate, review, and re-import. Never migrate on an unvalidated mapping set.

## Delta pass re-copies everything, or misses changes

**Symptom:** the delta wave transfers far more (or far fewer) items than the changes made on the source since pre-stage.

- **Cause:** delta/change-detection configuration or timing is off for your version.
- **Resolve:** confirm the delta configuration and change-detection behavior in Google's docs **for your version**; ensure the pre-stage pass fully completed before the delta; re-run the delta for the affected scope. Use `post-migration-validation.py` to confirm the incremental count is expected.

## Incomplete / skipped / unsupported items

**Symptom:** validation shows an item-count shortfall beyond the parity threshold; scan/run logs list skipped or unsupported items.

- **Cause:** items of a type/attribute Google Workspace Migrate does not carry, oversized objects, or items that errored mid-transfer.
- **Resolve:** open the validation report from `post-migration-validation.py`, identify the specific missing items, and disposition them: re-run the failed items, handle oversized objects per Google's supported-scope guidance **for your version**, and record documented unsupported types so the parity threshold accounts for them. Do not close the wave until the shortfall is explained.

## Node platform / database issues

**Symptom:** a worker node shows unhealthy/disconnected; throughput collapses; console errors on the platform node.

- **Cause:** node resource exhaustion (CPU/RAM/disk), the platform **database volume** near capacity, or a node lost connectivity to the platform node.
- **Resolve:** check node health in the console; free/expand the database volume (R1); restart the affected `GWSM Worker Node – Migration – NN` and confirm it re-registers to `GWSM Platform Node – Migration – Prod`. If throughput is structurally short of plan, add worker nodes per

`node-sizing-assessment.py`.

### Bandwidth / network egress bottleneck

**Symptom:** throughput is flat well below the plan even with nodes healthy and no throttling.

- **Cause:** the corporate egress bandwidth or a proxy/firewall path is the cap (R4).
- **Resolve:** confirm firewall/proxy allow-listing for the current endpoint list, measure available

sustained bandwidth vs. volume ÷ window, schedule bulk passes off-peak, or relocate nodes closer to better egress. Node count cannot exceed what bandwidth allows.

### Mail routing / cutover problems

**Symptom:** after cutover, mail loops, bounces, or does not deliver to the target mailbox; the mail round-trip test (T9) fails.

- **Cause:** mail routing/DNS not switched correctly, or coexistence routing conflict during cutover.
- **Resolve:** this is the high-impact step — execute the **cutover rollback**: revert mail routing to

the source under the same change window (GR-3/R7), keep users on the source, and diagnose the routing/DNS change before re-attempting. Source data is preserved through cutover, so the wave can be re-run once routing is corrected.

### Permissions/sharing dropped or over-broadened on target

**Symptom:** validation (T8) shows ownership wrong, sharing lost, or **unexpected public/broad sharing** introduced.

- **Cause:** bridge/mapping resolved identities incorrectly, or a source sharing construct mapped to a

broader target scope.

- **Resolve:** correct `Bridge – Identity – Corp`/the mapping for the affected objects and re-run that

scope; treat any unexpected public sharing as a **fail** and remediate before closing the wave.

## Best Practices

Durable practices for deploying and running **Google Workspace Migrate** at scale. They complement the ordered steps in `03-implementation.md`.

### Platform & sizing

- **Size from measured data, not guesses.** Drive node count from the `node-sizing-assessment.py`

baseline (volume ÷ window ÷ per-node throughput), and **round up** for delta passes and headroom. Under-provisioning silently blows the timeline.

- **Isolate the platform database.** Put it on a dedicated, encrypted volume with backup; it holds

all project state, mappings, and scan results — losing it means re-scanning everything.

- **Confirm supported scope for your version first.** Before committing a wave plan, verify in Google's Google Workspace Migrate docs exactly which content types/attributes the tool carries and which it does not — design mappings around the documented boundary, don't discover it mid-wave.

## Connections & identity

- **Least privilege, dedicated identities.** Use a dedicated Entra app (`Connection – Microsoft 365 – Source`) and a dedicated Google service account (`Connection – Google Workspace – Target`) — not a person's admin account — with only the scopes the current version requires. Store secrets in a secret manager; **never** put keys in code, CSVs, or the repo.
- **Get the identity bridge right before scaling.** `Bridge – Identity – Corp` is what preserves ownership and sharing. Validate that every source identity resolves to exactly one target (`connection-mapping-generator.py --validate`) before the pilot, not after.

## Waves, throttling & sequencing

- **Pilot first, always.** Prove the full pre-stage → delta → cutover sequence on `Migration Wave – Finance – Pilot` and pass every gate before any production wave.
- **Pre-stage bulk, delta close, cutover last.** Move the bulk while users work on the source, carry changes with deltas, and keep cutover (the only high-impact step) inside a change window with a tested rollback.
- **Start throttling conservative, calibrate from the pilot.** Begin below the API ceilings; use the pilot's measured throughput (`wave-batch-builder.py --calibrate`) to raise concurrency safely. It is faster to run steadily under the limit than to trip throttling and back off repeatedly.
- **Schedule bulk passes off-peak.** Bandwidth is a hard cap; run the heavy pre-stage passes in off-peak windows and keep cutover deltas small.

## Validation & change control

- **Validate every wave, not just the pilot.** Run `post-migration-validation.py` and clear item-count, permission, and mail gates before closing any wave. "It looked done" is not parity.
- **Watch throttling live.** Keep `migration-health-check.py --watch` running through each wave so sustained throttling is caught and tuned early, not discovered in the validation report.
- **Every production change is reversible.** Mail routing, DNS, and cutover changes go through change control with a documented rollback; keep source data intact through cutover so any wave can re-run.
- **Preserve the baseline.** `baseline-source.json` is the source of truth for parity — capture it before migrating and don't overwrite it.

### Safety & hygiene

• **Safe-by-default automation.** The Python tools are read-only or `--dry-run` until `--apply`; keep it that way. Review generated mapping/wave artifacts before importing them into the console.

• **Decommission cleanly.** Only after the final wave validates: decommission the nodes and **revoke** the migration app registration and service account so the powerful cross-tenant access does not linger.

## Deliverables Summary

Everything included in this solution, on one page.

### Documentation

• **Solution Overview** (`01-solution-overview.md`) — the Google Workspace Migrate architecture (platform/worker nodes, database, connections, mappings, bridges), where it fits versus the other migration solutions, the wave model, and the Assess → Design → Configure → Pilot → Validate → Expand deployment model.

• **Requirements Guide** (`02-requirements.md`) — solution-specific prerequisites (node platform, M365 app registration + scopes, Google Cloud project + delegated service account, network egress, source inventory, mapping/bridge data, service identities), each with how to verify/implement, building on the shared Google global requirements.

• **Implementation Guide** (`03-implementation.md`) — exact ordered steps: baseline, size the platform, build connections and mappings, scan, run the pilot wave (pre-stage → delta → cutover), validate, then expand — with pass/fail gates and documented rollback. Tool-console internals deferred to Google's docs; procedure and sequencing given.

• **Testing & Validation** (`05-testing.md`) — ten tests (T1–T10) mapped to the gates, each with an explicit pass/fail result.

• **Troubleshooting** (`06-troubleshooting.md`) — common failure modes (throttling/API limits, connection/permission errors, delta/incomplete-item issues, mapping/bridge errors, mail-flow) and resolutions.

• **Best Practices** (`07-best-practices.md`) — durable practices for platform sizing, connections/identity, waves/throttling/sequencing, validation, and change control.

• **Executive Summary** (`executive-summary.md`) — outcome, approach, and why it is low-risk.

### Python tools (5) — Google Workspace Admin SDK / Microsoft Graph, safe-by-default

1. `node-sizing-assessment.py` — read-only source discovery (Microsoft Graph): inventories

mailbox/OneDrive/SharePoint counts, sizes, largest objects, and sharing complexity, and recommends a worker-node count for a target throughput and window. Produces `baseline-source.json`, the validation reference.

1. `connection-mapping-generator.py` — generates and validates the **mapping CSVs** and the

**identity bridge** (`Bridge – Identity – Corp`) Google Workspace Migrate consumes, from directory data (Admin SDK + Graph). Dry-run/validate by default; never writes to a live system.

1. `wave-batch-builder.py` — groups in-scope users/sites into **waves**

(`Migration Wave – <Scope> – <Stage>`) and emits a **throttling plan** (concurrency, rate ceilings) sized to stay under source/target API limits; `--calibrate` retunes from measured pilot throughput.

1. `migration-health-check.py` — checks connection health and **watches a running wave** for

throughput, error rate, and sustained **throttling** (Graph 429 / Google rate limits), signaling when to back off. Read-only.

1. `post-migration-validation.py` — post-wave **validation report**: target-vs-baseline

item-count parity, permission/ownership fidelity, and mail count + flow checks (Admin SDK / Drive / Gmail read APIs). Read-only.

All scripts require Python 3.10+, use OAuth / ADC (no hardcoded credentials, no keys committed), take `--help`, and gate any write behind `--apply` (default is read-only or `--dry-run`).

## Packaged outputs

- `GWMIGRATE-Solution.pdf` — the compiled solution document (all sections above).
- `GWMIGRATE-Scripts.zip` — the five Python tools.

# Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

### connection-mapping-generator.py

### migration-health-check.py

### node-sizing-assessment.py

### post-migration-validation.py

### wave-batch-builder.py